

Clase 3:

Sistemas Recomendadores

Vicente Domínguez

Diplomado en Machine Learning Aplicado UC



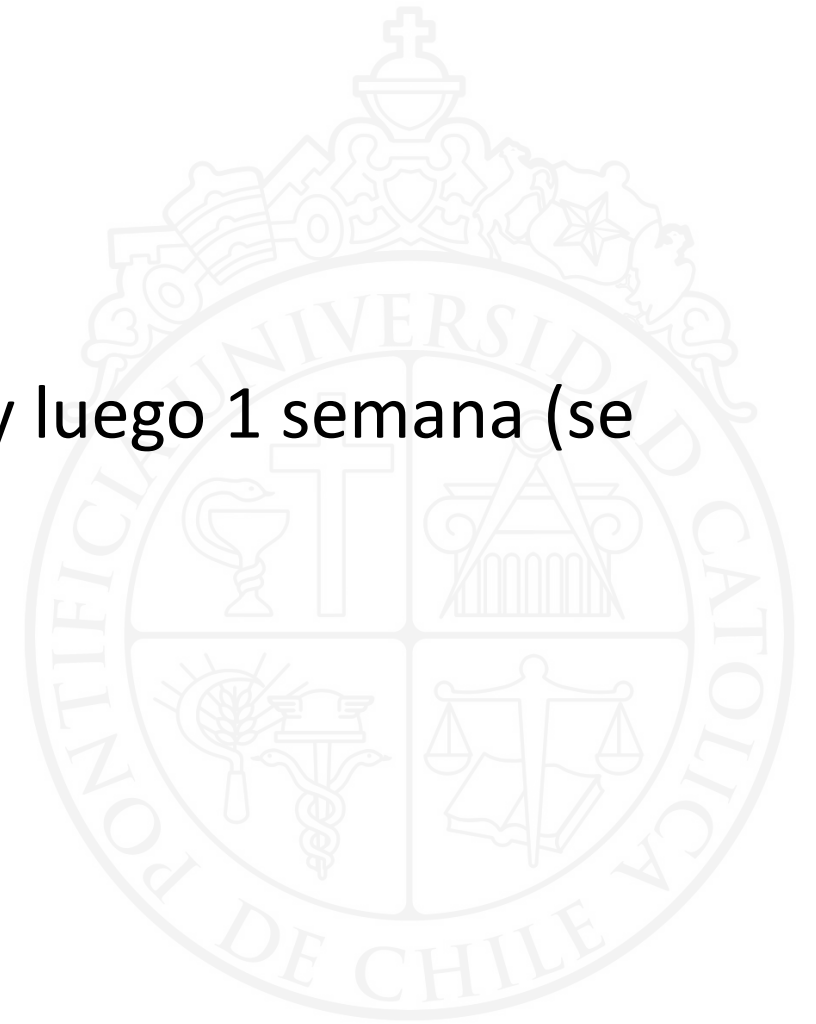
Programa del Curso

1. Introducción , Recomendación No personalizada , Filtrado Colaborativo (User y Item), Evaluación basada en métricas de error (RMSE, MAE) + **Práctico**.
2. Slope One , Recomendación basada en métodos latentes (Factorización Matricial) + Práctico.
3. Recomendación basada en feedback implícito , Optimización basada en ALS y BPR, Evaluación basada en métricas de ranking, novedad, serendipia + **Práctico**.
4. Recomendación basada en contenido (reglas de asociación, metadata y texto) + **Práctico**.
5. Recomendación basada en contexto, ensembles y recomendación híbrida. + **Práctico**.
6. Deep Learning para recomendación (texto , imágenes, multimodal, secuencial) + **Práctico**.
7. Reinforcement Learning (Bandits) + Conversational Recsys + **Práctico**
8. Repaso general , dominios de aplicación e investigación reciente , recomendación basada en grafos.

Evaluación

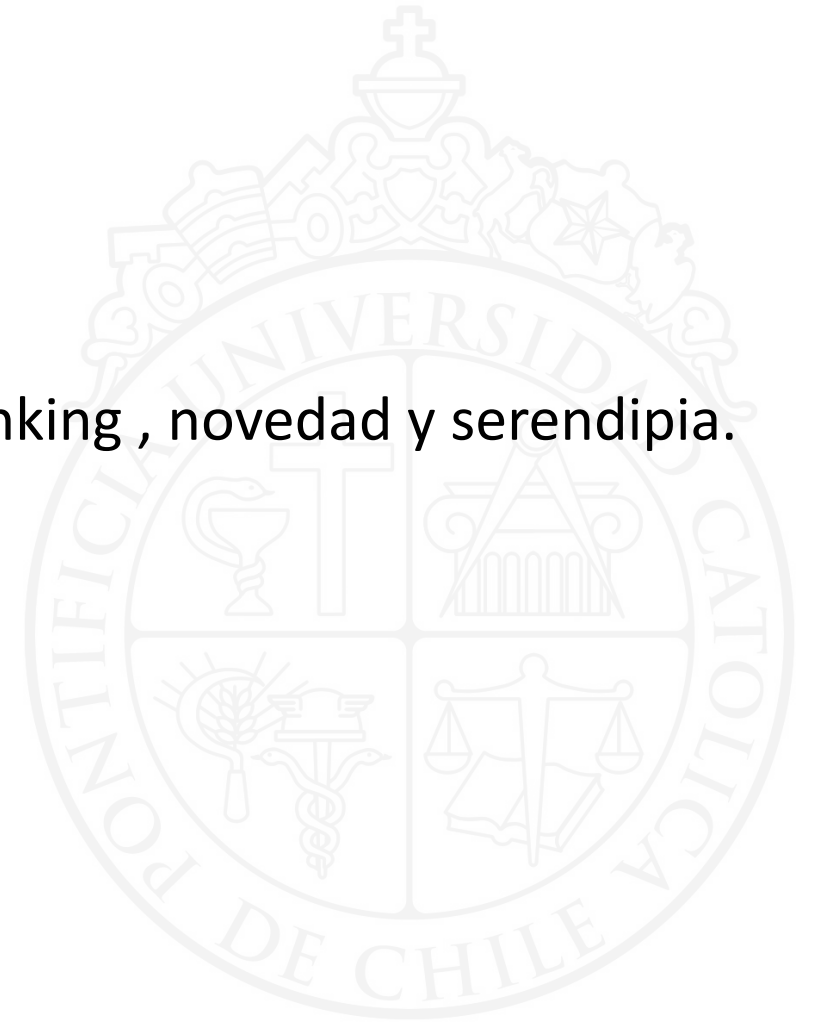
Promedio ejercicios prácticos.

Tiempo para realizarlos. Al final de la clase y luego 1 semana (se entrega en la siguiente clase)



En esta clase

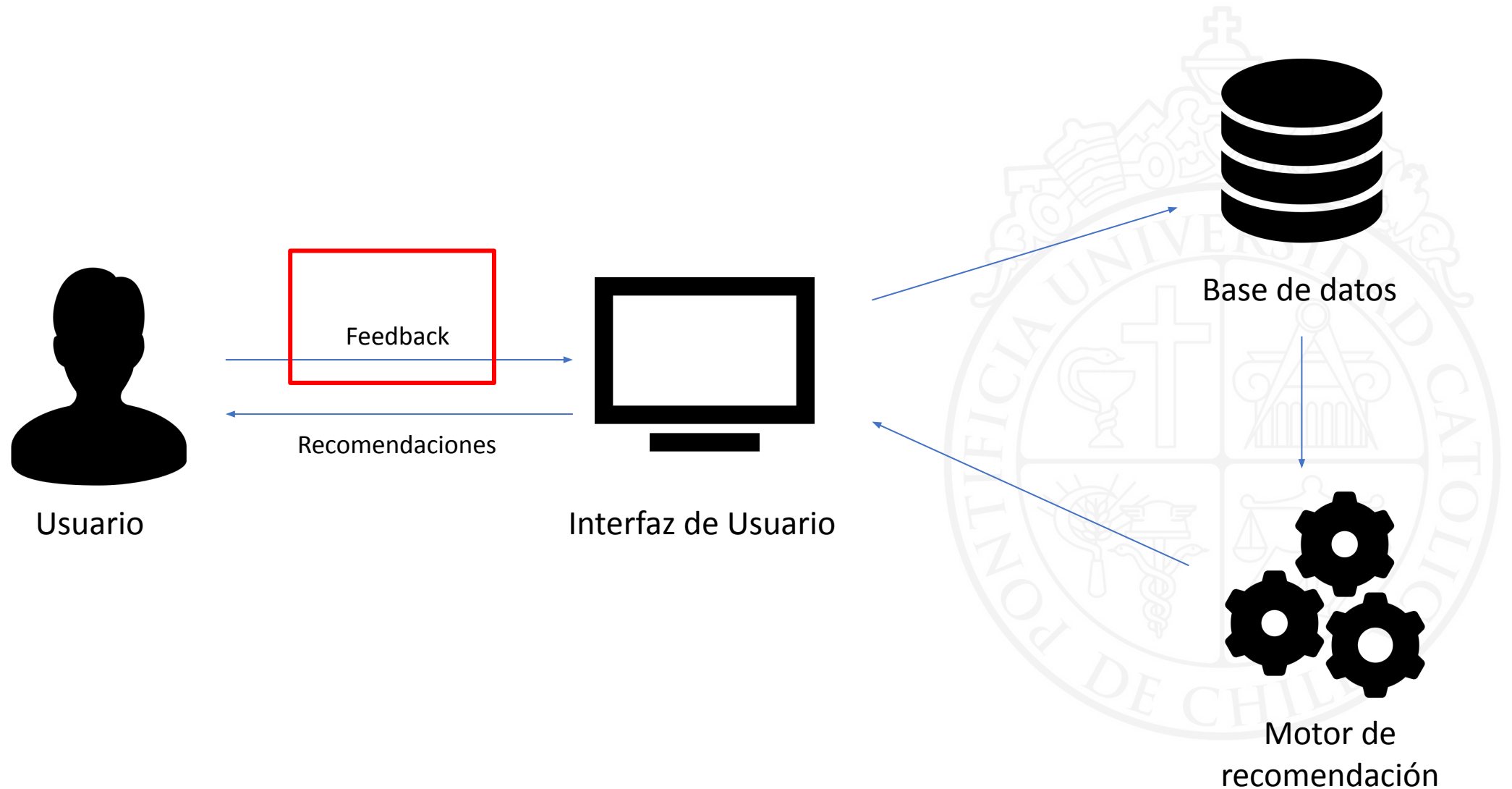
1. Repaso general.
2. Feedback implícito.
3. Optimización basada en ALS y BPR.
4. Evaluación de recomendación con métricas de ranking , novedad y serendipia.



Repaso

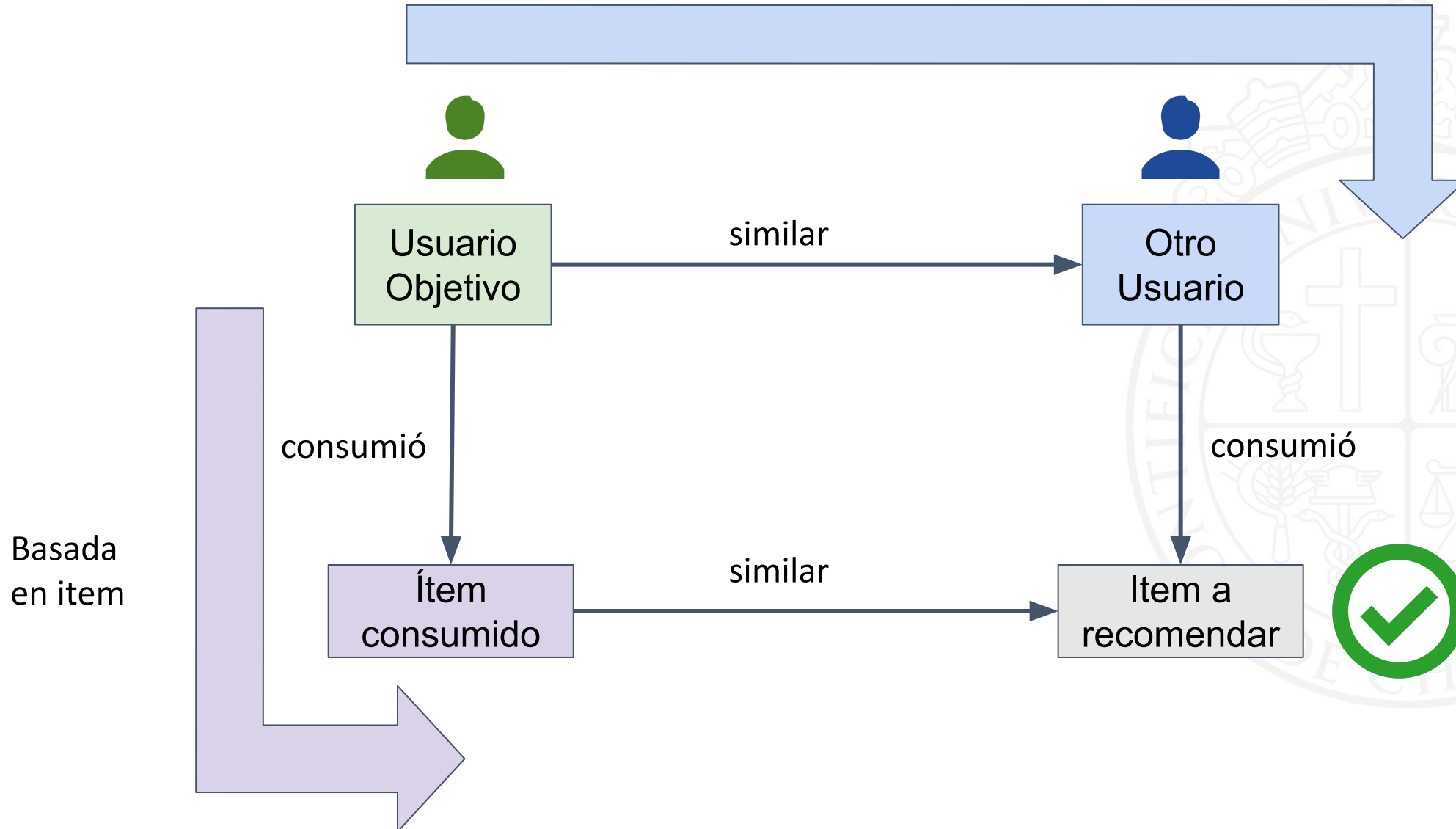


Esquema de recomendación



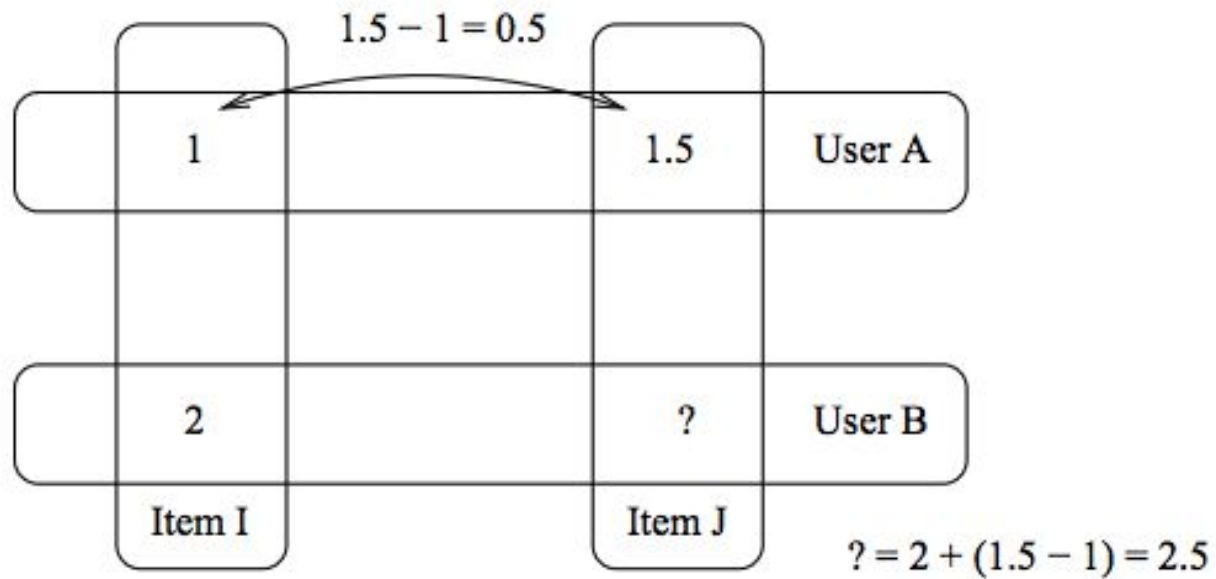
Filtrado colaborativo (resumen)

Basada en usuario

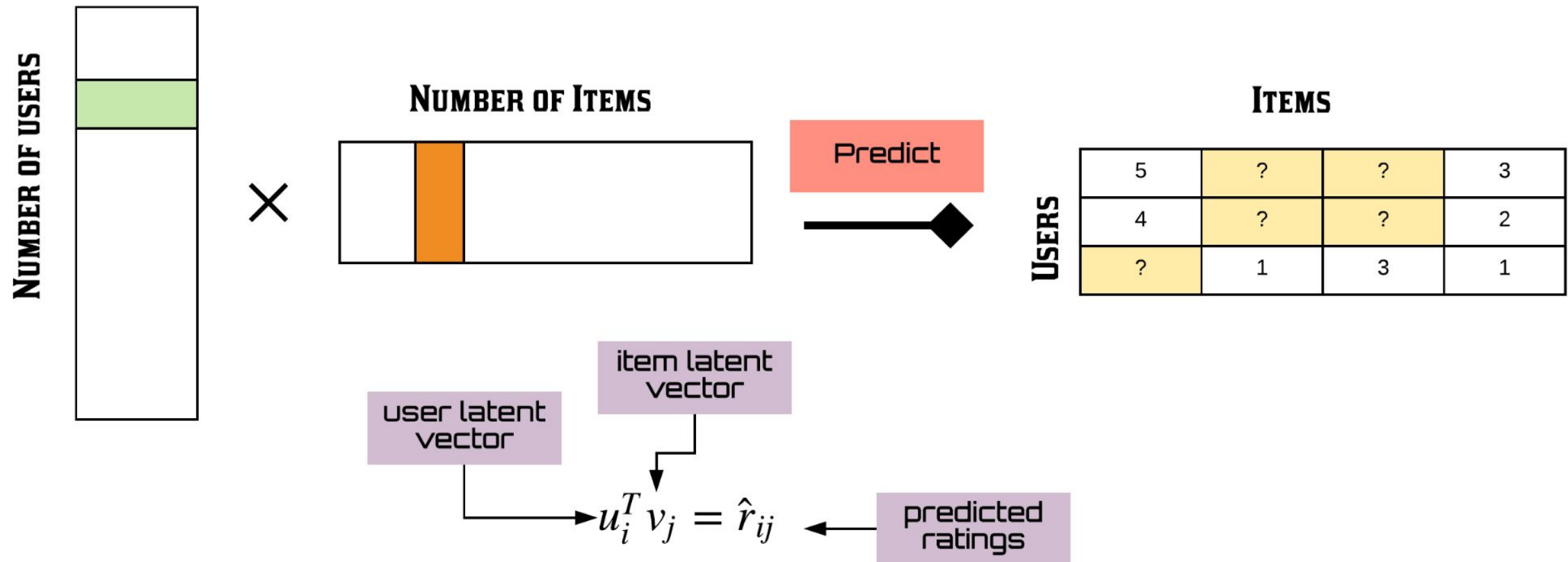


Slope One: intuición

- Diferencias de *ratings* entre pares de ítems generan una buena predicción



Recomendación basada en factores latentes (factorización matricial SVD).



Factorización Matricial - *FunkSVD*

- Una solución es el modelo *FunkSVD* y consiste en solucionar el siguiente modelo de optimización (<https://sifter.org/simon/journal/20061211.html>)

$$\min_{p^*, q^*} \sum_{(u,i) \in K} (r_{ui} - q_i^T \cdot p_u)^2 + \lambda(\|q_i\|^2 + \|p_u\|^2)$$

donde

r_{ui} : rating que asignó el usuario u al ítem i

q_i : vector latente del ítem i

p_u : vector latente del usuario u

λ : constante de regularización

Reflexión

Hasta ahora hemos evaluado una recomendación basada en el error de predicción de ratings (RMSE).

¿Cómo podemos hacer uso de feedback implícito (sólo interacciones)?

¿Cómo evaluamos la capacidad del recomendador para rankear ítems relevantes para un usuario objetivo?

Feedback explícito vs feedback implícito





Valerie

★★★★★ **Muy feliz**

Calificado en Estados Unidos 🇺🇸 el 15 de abril de 2023

Tamaño: L2 | **Compra verificada**

Nunca he jugado mucho al tenis, pero he tenido una pareja a la que le gusta jugar y una nueva oportunidad de jugar. Tenía unas cuantas raquetas más baratas aquí, Head and Wilson, ya que esta raqueta es una mejora definitiva para mí. Yo lo recomendaría.



Betty

★★★★☆ **Principiante**

Calificado en Estados Unidos 🇺🇸 el 7 de marzo de 2023

Tamaño: L2 | **Compra verificada**

La raqueta tiene colores reservados, pero es para personas muy principiantes, es buena opción para tenerla en el bolso y prestársela a algún amigo que no juegue tenis diariamente

Llamamos retroalimentación explícita a aquellas acciones donde el usuario nos indica directamente sus preferencias, por ejemplo las calificaciones o ratings, y las acciones "pulgar arriba"



Llamamos **retroalimentación implícita** a aquellas acciones o señales del usuario que podríamos interpretar como preferencia pero con cierto grado de incertidumbre: clicks, tiempo revisando la página de un producto, cantidad de visitas,

Características de la retroalimentación implícita

1. Es más fácil de obtener que la retroalimentación explícita. No siempre la gente quiere dar su opinión a través de calificaciones o ratings.
2. La RI no contiene esencialmente información de preferencias negativas, a diferencia de calificaciones o ratings explícitos.
3. La RI contiene ruido y es difícil cuantificar preferencias de los usuarios y confianza sobre esas preferencias.
4. No podemos usar métricas como RMSE o MAE para medir el rendimiento, debemos usar métricas de ranking

Optimización de recomendación basada en factorización n feedback implícito

1. Alternate Least Squares (ALS / WRMF) para feedback implícito.
2. Bayesian Personalized Ranking (BPR)



Retroalimentación Implícita o Implicit Feedback

- Hu, Y., Koren, Y., & Volinsky, C. (2008) modificaron el modelo de SVD para incluir *feedback* implícito.

Binarización de los datos

$$p_{ui} = \begin{cases} 1 & \text{si } r_{ui} > 0 \\ 0 & \text{si } r_{ui} = 0 \end{cases}$$

- interacción

Función de confianza

$$c_{ui} = 1 + \alpha r_{ui}$$

- definido por un alpha.
- valor mínimo de confianza = 1

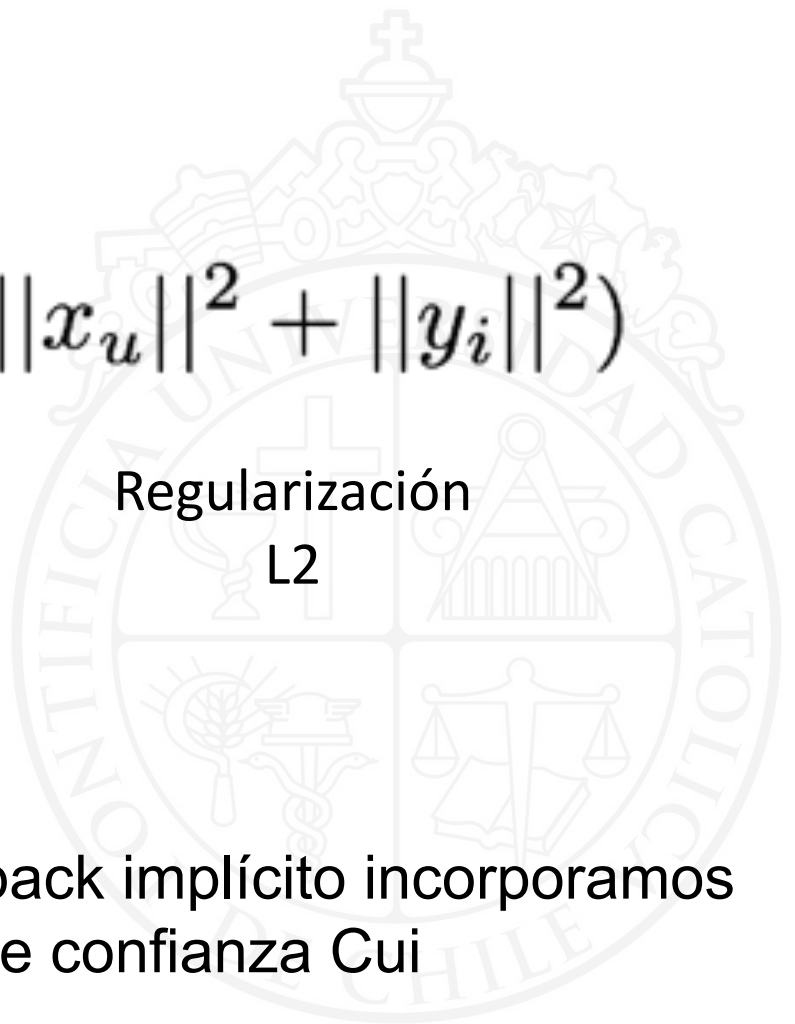
matrices de feedback implícito

P_{ui}

	item 1	item 2	item 3	
usuario 1	1	0	0	
usuario 2	0	1	1	
....				

C_{ui}

	item 1	item 2	item 3	
usuario 1	1.1	0	0	
usuario 2	0	1.5	1.02	
....				


$$\min_{x^*, y^*} \sum_{u, i} c_{ui} (p_{ui} - x_u^T y_i)^2 + \lambda (||x_u||^2 + ||y_i||^2)$$

Predicción de
Factorización
Matricial

Regularización
L2

Factor de confianza del
usuario u en el item i

Para feedback implícito incorporamos
un factor de confianza Cui

dataset 1

Transacciones en una tienda:

	CustomerID	StockCode	Description	Quantity
0	12346.0	23166	MEDIUM CERAMIC TOP STORAGE JAR	1
1	12347.0	16008	SMALL FOLDING SCISSOR(POINTED EDGE)	24
2	12347.0	17021	NAMASTE SWAGAT INCENSE	36
3	12347.0	20665	RED RETROSPOT PURSE	6
4	12347.0	20719	WOODLAND CHARLOTTE BAG	40

dataset 2

Transacciones en un restaurant con información de categoría , cocina, restaurant , cantidad de compras.

	user_id	item_id	dow	hod	category_id	cusine_id	restaurant_id	item_count
0	-6088859261566846925	601d4dc5-2ad	7	3	9c64431e-79f	fb8b5b7f-013	706247b3-86e	1
1	-4662982070704311223	af48a23f-f80	7	2	a989cb37-db5	8631ea96-a78	ea90883c-a2f	1
2	-4849981472186386714	bab8cfae-930	1	1	4f31b9ea-059	5aa1a680-942	d9ab8a13-380	1
3	-5400975689533971605	3ae9d553-282	4	0	37d510f8-745	c9aa3993-86f	0b8d021a-c5c	1
4	4810905585891548302	3d673534-8f8	3	1	4264559d-470	5aa1a680-942	5772716f-1f3	1

dataset 3

Transacciones de productos bancarios con tiempo de duración y montos

usuario	producto bancario	tiempo (meses)	monto
u19	crédito hipotecario	240	15000 UF
u19	crédito consumo	60	150 UF
u23	fondo mutuo	80	1200 UF

Optimización de recomendación basada en factorización matricial con feedback implícito

1. Alternate Least Squares (ALS)
2. Bayesian Personalized Ranking (BPR)



Retroalimentación Implícita o Implicit Feedback

- Hu, Y., Koren, Y., & Volinsky, C. (2008) modificaron el modelo de SVD para incluir *feedback* implícito

Binarización de los datos

$$p_{ui} = \begin{cases} 1 & \text{si } r_{ui} > 0 \\ 0 & \text{si } r_{ui} = 0 \end{cases}$$

Ej. lo compra o no lo compra?
interacción?

Función de confianza

$$c_{ui} = 1 + \alpha r_{ui}$$

Ej. - dwell time (más tiempo , mayor score)
- monto \$, UF

Función de pérdida

$$\min_{x^*, y^*} \sum_{u, i} c_{ui} (p_{ui} - x_u^T y_i)^2 + \lambda (||x_u||^2 + ||y_i||^2)$$

Optimización de recomendación basada en factorización matricial con feedback implícito

1. Alternate Least Squares (ALS)
2. Bayesian Personalized Ranking (BPR)



Filtrado Colaborativo con Retroalimentación Implícita

El artículo “Collaborative Filtering for Implicit Feedback Datasets” de Hu, Koren y Volinsky introduce el método de factorización matricial ALS / WRMF (Alternating Least Squares).

Collaborative Filtering for Implicit Feedback Datasets

Yifan Hu
AT&T Labs – Research
Florham Park, NJ 07932

Yehuda Koren*
Yahoo! Research
Haifa 31905, Israel

Chris Volinsky
AT&T Labs – Research
Florham Park, NJ 07932

Filtrado Colaborativo con Retroalimentación Implícita

El modelo ALS permite:

1

Aprender representaciones latentes de usuarios e ítems a partir de feedback implícito como clicks o cantidad de reproducciones de una canción o serie de TV.

2

Escalar la cantidad de usuarios e ítems en el modelo al paralelizar el entrenamiento con Mínimos Cuadrados Alternados (ALS en inglés)

3

Evaluar el modelo propuesto en una caso real, como un proveedor de TV digital.

Modelamiento de los clicks de usuarios

Al solo contar con “cantidad de veces que se ve un producto” y no con calificaciones explícitas, Hu et al. modelaron el entrenamiento de manera distinta a FunkSVD

1

Introducen una variable de preferencia p_{ui} que solo toma valores 1 o 0

$$p_{ui} = \begin{cases} 1 & r_{ui} > 0 \\ 0 & r_{ui} = 0 \end{cases}$$

donde r_{ui} indica cuántas veces el usuario u consumió el producto i

Modelamiento de confianza sobre las preferencias

Otro concepto que introduce el método de Hu et al. es el de confianza en la retroalimentación implícita. Como no es una señal explícita como las calificaciones en forma de 1 a 5 estrellas, se incorpora el concepto de confianza.

2

Introducen la variable c_{ui} que mide la confianza en observar la preferencia p_{ui} y puede tomar las siguientes 2 formas:

$$c_{ui} = 1 + \alpha r_{ui}$$

$$c_{ui} = 1 + \alpha \log(1 + r_{ui}/\epsilon).$$

donde r_{ui} indica cuántas veces el usuario u consumió el producto i , y α es un hiperparámetro que se puede optimizar posteriormente, que indica la tasa a la que crece la confianza que se tiene en la preferencia a medida que aumenta el consume de i

Función de pérdida de ALS

Finalmente, la nueva función que espera encontrar los vectores latentes x_* e y_* que minimizan la pérdida es:

$$\min_{x_*, y_*} \sum_{u, i} c_{ui} (p_{ui} - x_u^T y_i)^2 + \lambda \left(\sum_u \|x_u\|^2 + \sum_i \|y_i\|^2 \right)$$

p_{ui} : Preferencia del usuario u por el ítem i (1 / 0)

c_{ui} : confianza en la preferencia del usuario

x_u : vector de factores latentes del usuario

y_i : vector de factores latentes del ítem

λ : coeficiente de regularización

Paralelización del cálculo

El factor latente de un usuario u se calcula a continuación, donde la matriz Y es la matriz de preferencias usuario-ítem y la matriz C^u es la matriz de “confianzas” de los ítems consumidos por el usuario u

$$x_u = (Y^T C^u Y + \lambda I)^{-1} Y^T C^u p(u)$$

El factor latente de un ítem i se calcula a continuación, donde la matriz X es la matriz de preferencias de todos los ítems y la matriz C^i es la matriz de “confianzas” de ítems con ítems.

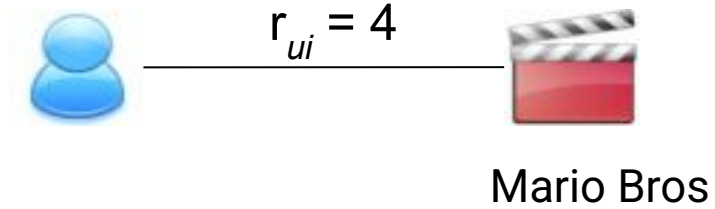
$$y_i = (X^T C^i X + \lambda I)^{-1} X^T C^i p(i)$$

Optimización de recomendación basada en factorización matricial con feedback implícito

1. Alternate Least Squares (ALS)
2. Bayesian Personalized Ranking (BPR)



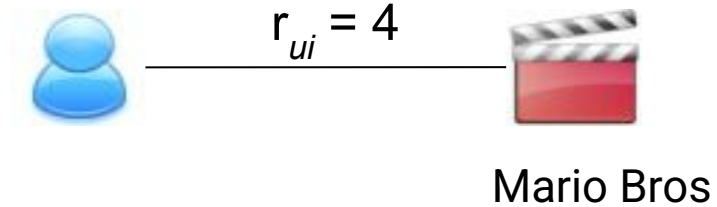
Tipos de aprendizaje



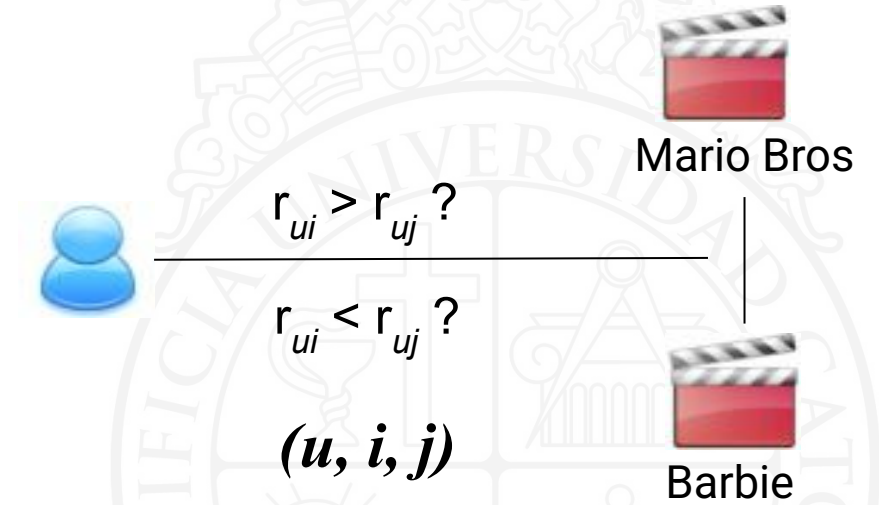
**Aprendizaje de preferencia
directa**
(point-wise learning)



Tipos de aprendizaje

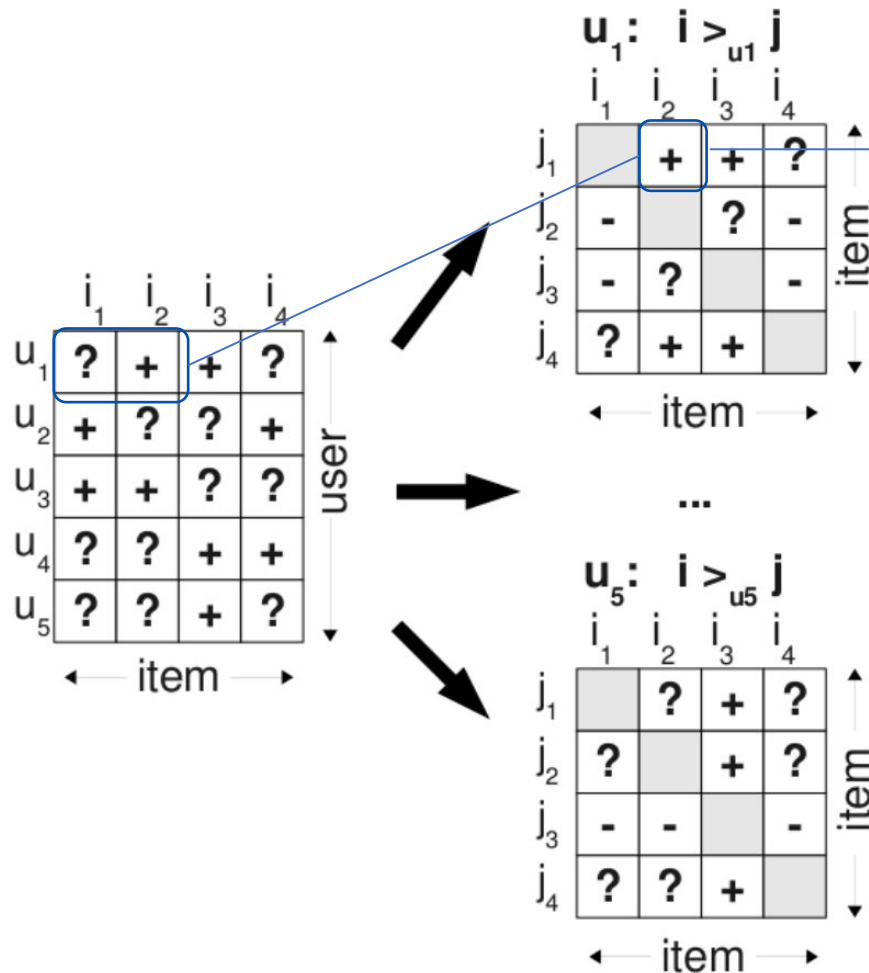


**Aprendizaje de preferencia
directa**
(point-wise learning)



**Aprendizaje de preferencia por
comparación de pares**
(pairwise-wise learning)

Transformando retroalimentación en positiva y negativa



$$\mathcal{D}_p = \{(u, i, j) \in \mathcal{D} | i \in \mathcal{I}_u \wedge j \in \mathcal{I} \setminus \mathcal{I}_u\}$$

BPR

BPR significa “Bayesian Personalized Ranking” :

1. Su objetivo es encontrar una función de ranking personalizado.
2. Uno de los métodos de “Aprender a rankear” más populares.
3. BPR por sí mismo no es un algoritmo: más bien una función de pérdida y un framework para llevar a cabo la optimización.
4. Algoritmo = modelo + función de pérdida + aprendizaje

Ejemplo del algoritmo BPR-MF

BPR-MF es un algoritmo que usa el framework de BPR con factorización matricial

1. Modelo: MF (factorización matricial)
2. Función de pérdida: BPR-OPT
3. Aprendizaje: BPR-Learn (basado en SGD)



¿Cómo funciona BPR?

Los datos de entrenamiento son tuplas de la forma:

(u, i, j)

Que indica que el **usuario** u prefiere el **ítem** i por sobre el **ítem** j

**BPR intenta clasificar el ítem positivo más alto que el ítem negativo.
Este proceso proporciona un ranking personalizado para cada usuario.**

SUPUESTO FUERTE: items que aún no han sido consumido son menos preferidos.

Optimización con BPR (BPR-opt)

BPR utiliza una pairwise loss function y modela la probabilidad de que el usuario u prefiera el ítem i sobre el ítem j

Factorización matricial de vectores de ítems preferidos ($\mathbf{V}_i$) y no preferidos ($\mathbf{V}_j$) por el usuario U . U , $\mathbf{V}_i$ y $\mathbf{V}_j$ comienzan aleatorios.

$\mathbf{X}_{ui} = U \times \mathbf{V}_i^T$ (preferidos)

$\mathbf{X}_{uj} = U \times \mathbf{V}_j^T$ (no preferidos)

Diferencia entre resta de los resultados anteriores:

$\mathbf{X}_{uij} = \mathbf{X}_{ui} - \mathbf{X}_{uj}$

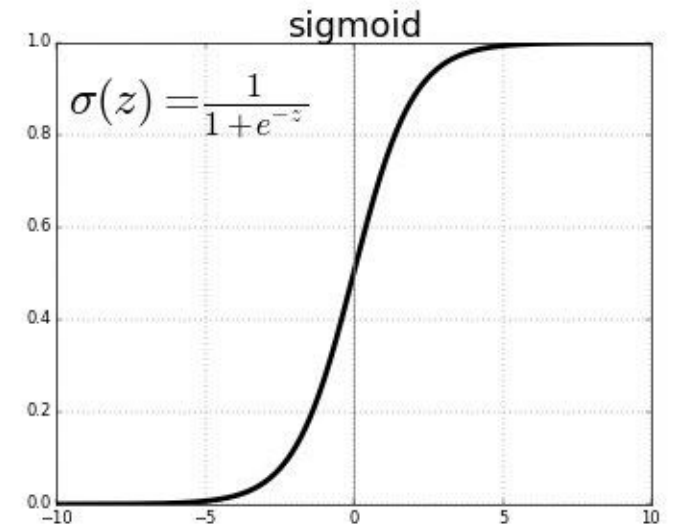
Función de pérdida:

BPR-opt = - log_prob + regularización

Donde:

$\text{log_prob} = \sum (\log(\text{sigmoid}(\mathbf{X}_{uij})))$

$\text{regularización} = \lambda * (\sum (||U||^2) + \sum (||\mathbf{V}_i||^2) + \sum (||\mathbf{V}_j||^2))$



Función de pérdida BPR

Recordemos la función de pérdida, donde $\hat{\phi}_{ui}$ corresponde a la predicción de preferencia del usuario u por el ítem i :

$$\arg \max_{\Theta} \underbrace{\sum_{(u,i,j) \in \mathcal{D}_p} \ln \sigma(\hat{\phi}_{ui} - \hat{\phi}_{uj}) - \lambda \|\Theta\|^2}_{BPR-OPT}$$

De esta ecuación nos interesa a continuación aprender los parámetros Θ que maximizan la expresión BPR-OPT, que no son otra cosa que los vectores latentes de usuarios y de ítems.

Aprendizaje de parámetros con BPR



BPR-Learn: Maximizar BPR-OPT

Para encontrar los valores de los parámetros Θ , derivamos BPR-OPT respecto a los parámetros

$$\begin{aligned}\frac{\partial \text{BPR-OPT}}{\partial \Theta} &= \sum_{(u,i,j) \in D_S} \frac{\partial}{\partial \Theta} \ln \sigma(\hat{x}_{uij}) - \lambda_{\Theta} \frac{\partial}{\partial \Theta} \|\Theta\|^2 \\ &\propto \sum_{(u,i,j) \in D_S} \frac{-e^{-\hat{x}_{uij}}}{1 + e^{-\hat{x}_{uij}}} \cdot \frac{\partial}{\partial \Theta} \hat{x}_{uij} - \lambda_{\Theta} \Theta\end{aligned}$$

y con esto podemos obtener la regla de actualización para aplicar SGD (Descenso de Gradiente Estocástico)

Reglas de Actualización

La siguiente ecuación nos indica la regla de actualización de parámetros

$$\Theta \leftarrow \Theta + \alpha \left(\frac{e^{-\hat{x}_{uij}}}{1 + e^{-\hat{x}_{uij}}} \cdot \frac{\partial}{\partial \Theta} \hat{x}_{uij} + \lambda_{\Theta} \Theta \right)$$

- Si recordamos la derivación de otros modelos como FunkSVD, veremos que se inicializan los valores de Θ de forma aleatoria, se fijan los valores de los hiperparámetros alfa y lambda, y se va actualizando de forma iterativa los valores de Θ .
- La forma final de la derivada de $\hat{x}_{uij}$ dependerá de cómo definamos el score de preferencia que el usuario u tiene por el ítem i al compararlo con el ítem j

Reglas de Actualización

BPR-Learn para BPR-MF

Si aplicamos el framework BPR a Factorización Matricial (BPR-MF), tendremos las siguientes reglas de actualización

- **Definimos** $\hat{x}_{uij} := \hat{x}_{ui} - \hat{x}_{uj}$

- **Donde $\hat{x}_{ui}$ equivale a producto punto de vectores latentes**

$$\hat{x}_{ui} = \langle w_u, h_i \rangle = \sum_{f=1}^k w_{uf} \cdot h_{if}$$

- **Luego, usando BPR-LEARN**

$$\Theta \leftarrow \Theta + \alpha \left(\frac{e^{-\hat{x}_{uij}}}{1 + e^{-\hat{x}_{uij}}} \cdot \frac{\partial}{\partial \Theta} \hat{x}_{uij} + \lambda_{\Theta} \Theta \right)$$

$$\frac{\partial}{\partial \theta} \hat{x}_{uij} = \begin{cases} (h_{if} - h_{jf}) & \text{if } \theta = w_{uf}, \\ w_{uf} & \text{if } \theta = h_{if}, \\ -w_{uf} & \text{if } \theta = h_{jf}, \\ 0 & \text{else} \end{cases}$$

Optimización con BPR (BPR-opt)

BPR utiliza una pairwise loss function y modela la probabilidad de que el usuario u prefiera el ítem i sobre el ítem j

Factorización matricial de vectores de ítems preferidos ($\mathbf{V}_i$) y no preferidos ($\mathbf{V}_j$) por el usuario U

$\mathbf{X}_{ui} = U \times \mathbf{V}_i^T$ (preferidos)

$\mathbf{X}_{uj} = U \times \mathbf{V}_j^T$ (no preferidos)

Diferencia entre resta de los resultados anteriores:

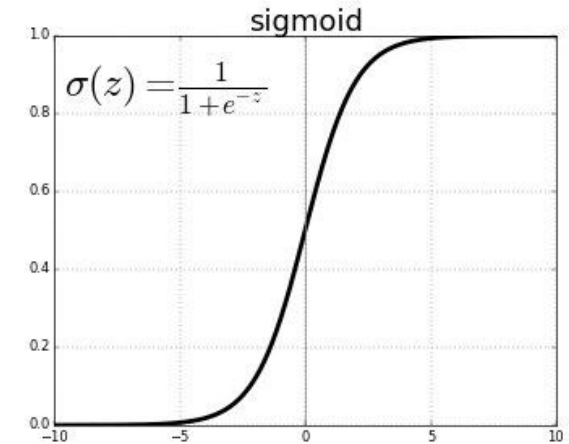
$\mathbf{X}_{uij} = \mathbf{X}_{ui} - \mathbf{X}_{uj}$

$\text{log_prob} = \text{sum}(\text{log}(\text{sigmoid}(\mathbf{X}_{uij})))$

$\text{regularización} = \text{lambda} * (\text{sum}(\|\mathbf{U}\|^2) + \text{sum}(\|\mathbf{V}_i\|^2) + \text{sum}(\|\mathbf{V}_j\|^2))$

Función de pérdida:

$\text{BPR-opt} = -\text{log_prob} + \text{regularización}$



Generación de recomendación con BPR

- BPR se entrena con datos de interacción del usuario para aprender de manera paralela los factores latentes de los ítems preferidos y no preferidos, así como los factores latentes de los usuarios.
- La preferencia predicha de un usuario para un ítem se calcula como el producto escalar de los factores latentes correspondientes al usuario e ítem.
- Las recomendaciones se generan ordenando los ítems según las preferencias predichas, es decir, los scores calculados.

Código en pytorch de BPR

$W[u,:]$, $W[i,:]$, $W[j, :]$ → busca índices de usuarios (u), ítems preferidos (i) e ítems no preferidos (j) en matriz W

```
class BPR(nn.Module):
    def __init__(self, user_size, item_size, dim, weight_decay):
        super().__init__()
        self.W = nn.Parameter(torch.empty(user_size, dim))
        self.H = nn.Parameter(torch.empty(item_size, dim))
        nn.init.xavier_normal_(self.W.data)
        nn.init.xavier_normal_(self.H.data)
        self.weight_decay = weight_decay

    def forward(self, u, i, j):
        u = self.W[u, :]
        i = self.H[i, :]
        j = self.H[j, :]
        x_ui = torch.mul(u, i).sum(dim=1)
        x_uj = torch.mul(u, j).sum(dim=1)
        x_uij = x_ui - x_uj
        log_prob = F.logsigmoid(x_uij).sum()
        regularization = self.weight_decay * (u.norm(dim=1).pow(2).sum() + i.norm(dim=1).pow(2).sum() + j.norm(dim=1).pow(2).sum())
        return -log_prob + regularization
```

Matriz H se actualiza de manera paralela con ítems preferidos (i) y no preferidos (j).

Recomendación

```
class BPR(nn.Module):
    def __init__(self, user_size, item_size, dim, weight_decay):
        super().__init__()
        self.W = nn.Parameter(torch.empty(user_size, dim))
        self.H = nn.Parameter(torch.empty(item_size, dim))
        nn.init.xavier_normal_(self.W.data)
        nn.init.xavier_normal_(self.H.data)
        self.weight_decay = weight_decay

    def forward(self, u, i, j):
        u = self.W[u, :]
        i = self.H[i, :]
        j = self.H[j, :]
        x_ui = torch.mul(u, i).sum(dim=1)
        x_uj = torch.mul(u, j).sum(dim=1)
        x_uij = x_ui - x_uj
        log_prob = F.logsigmoid(x_uij).sum()
        regularization = self.weight_decay * (u.norm(dim=1).pow(2).sum() + i.norm(dim=1).pow(2).sum() + j.norm(dim=1).pow(2).sum())
        return -log_prob + regularization
```

```
def recommend(self, u):
    u = self.W[u, :]
    x_ui = torch.mm(u, self.H.t())
    pred = torch.argsort(x_ui, dim=1)
    return pred
```

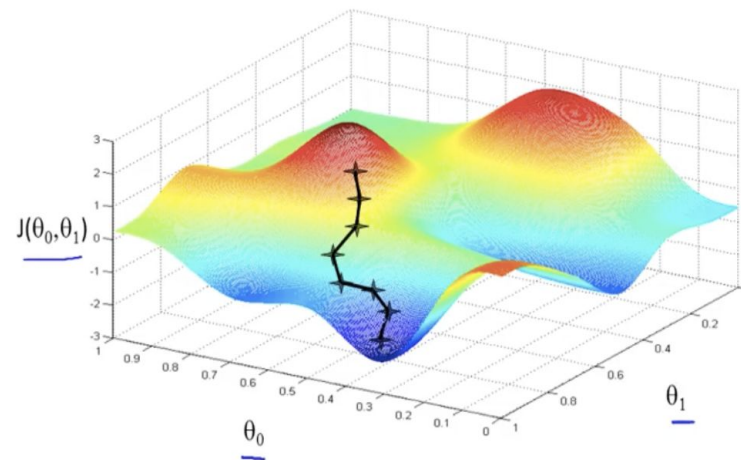
Recomendación:

Producto punto entre vectores aprendidos de usuario (W) y matriz de ítems (H) transpuesta (T).

Predicción final ordena items por similaridad.

Trucos de BPR

- Como son muchas combinaciones de (u,i,j) los autores hacen sampling aleatorio con repetición. (bootstrap sampling)
- La actualización de los factores latentes de usuario e item se actualizan con **descenso de gradiente**, buscando maximizar la distancia entre los preferidos sobre los no preferidos.



Evaluación de recomendador basado en feedback implícito.

Métricas de ranking.

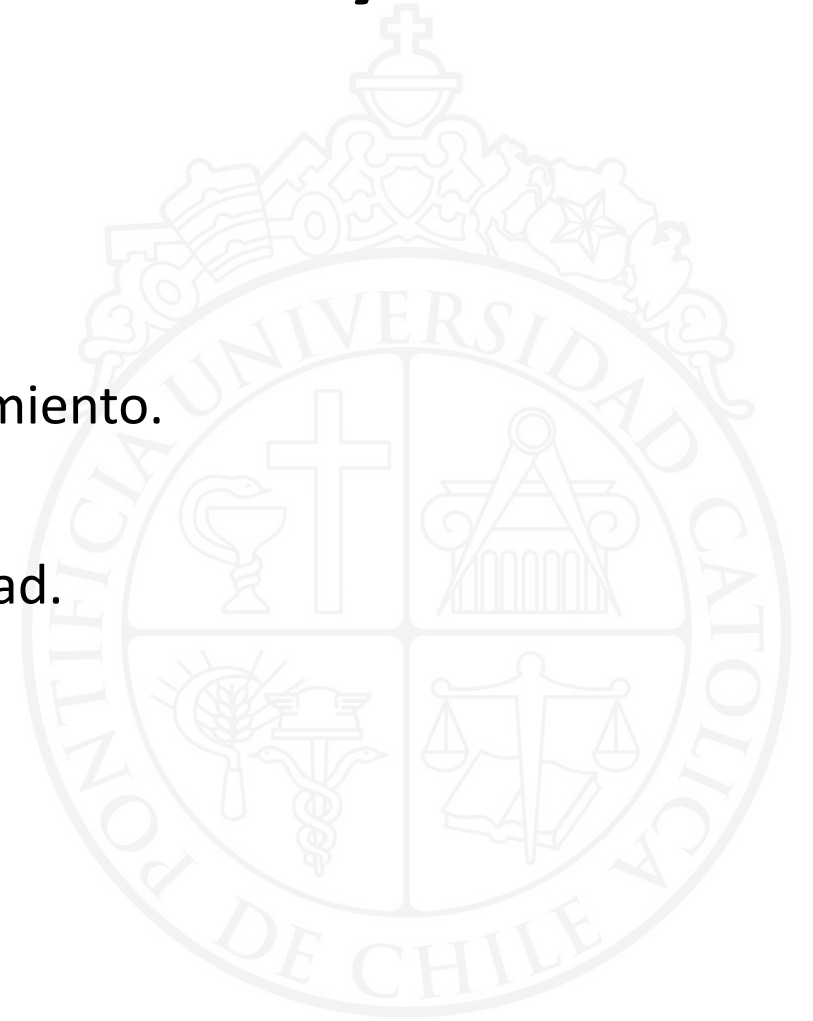


Partición de set de entrenamiento, validación y test

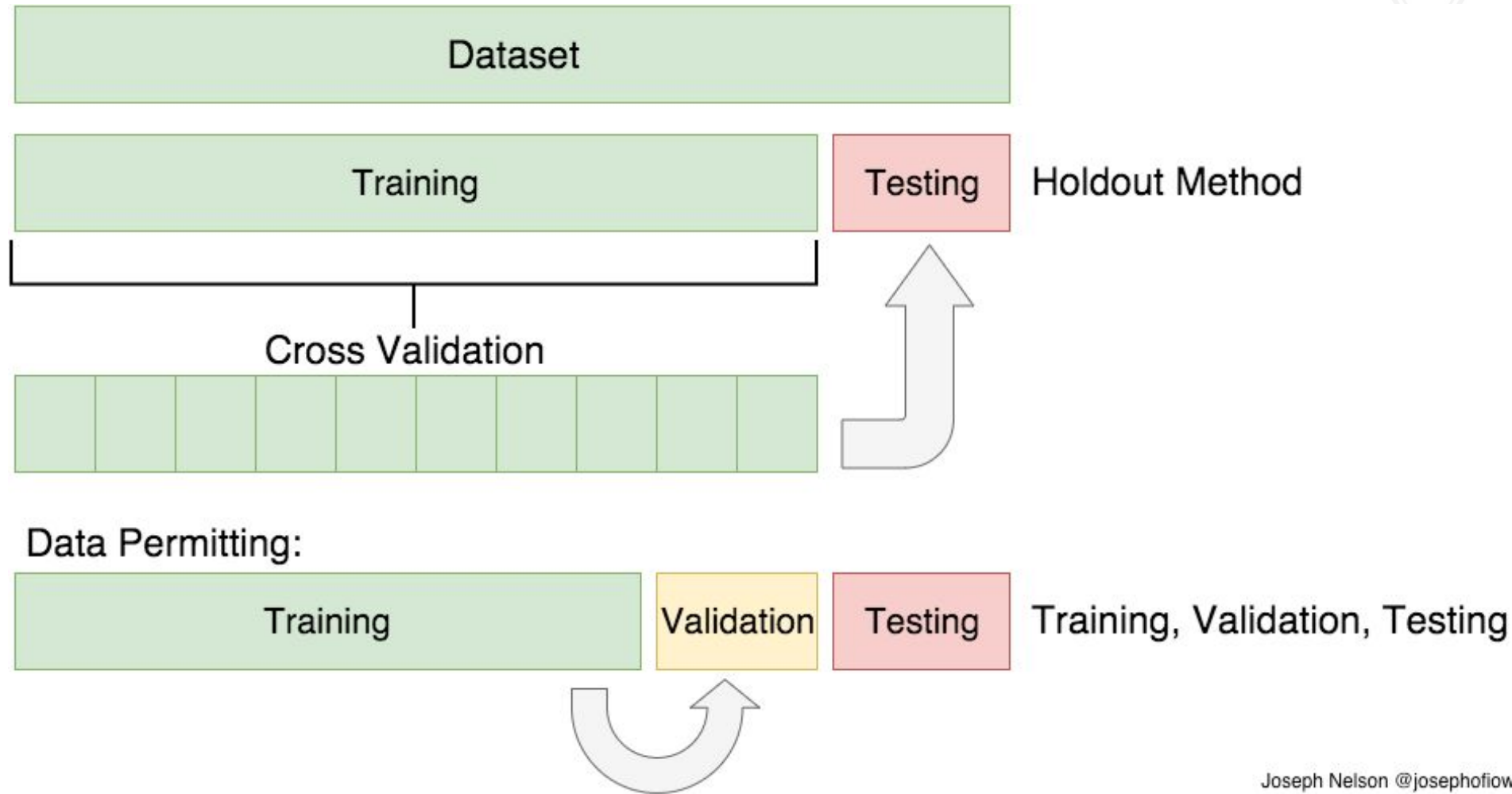
Set de entrenamiento: para entrenar el modelo.

Set de validación: para ver cómo está funcionando el entrenamiento.

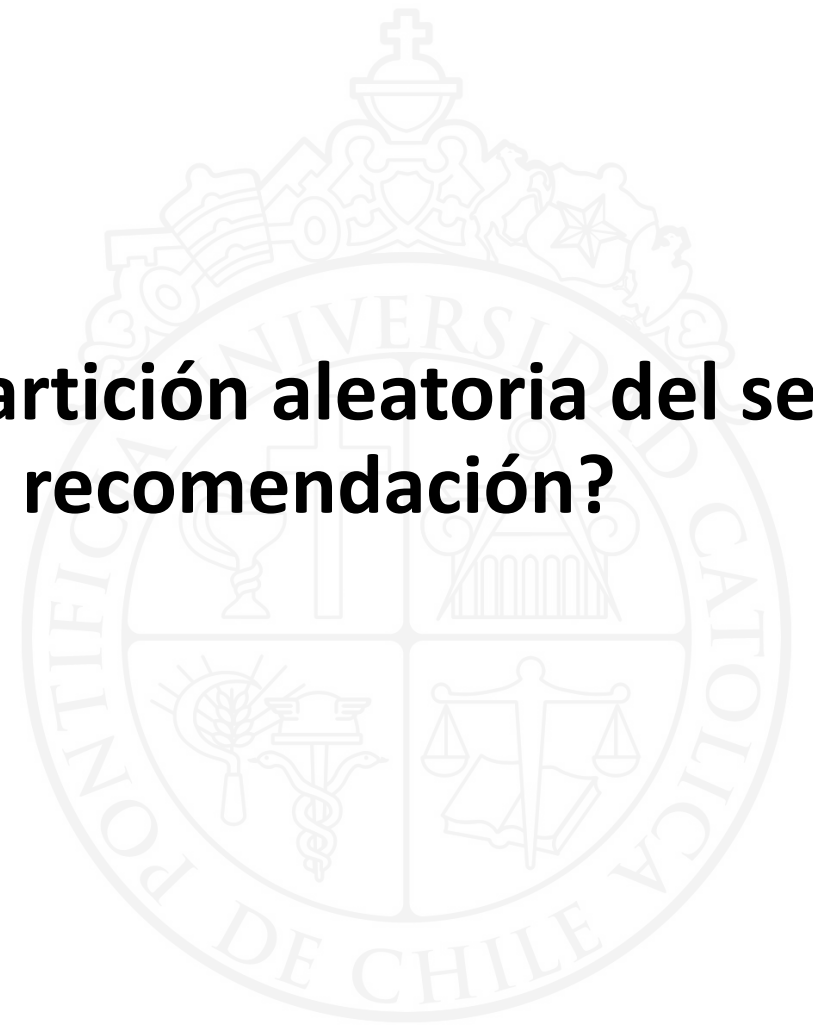
Set de test: para testear con datos lo más parecidos a la realidad.



Partición de set de train, validación y test (cross validation)

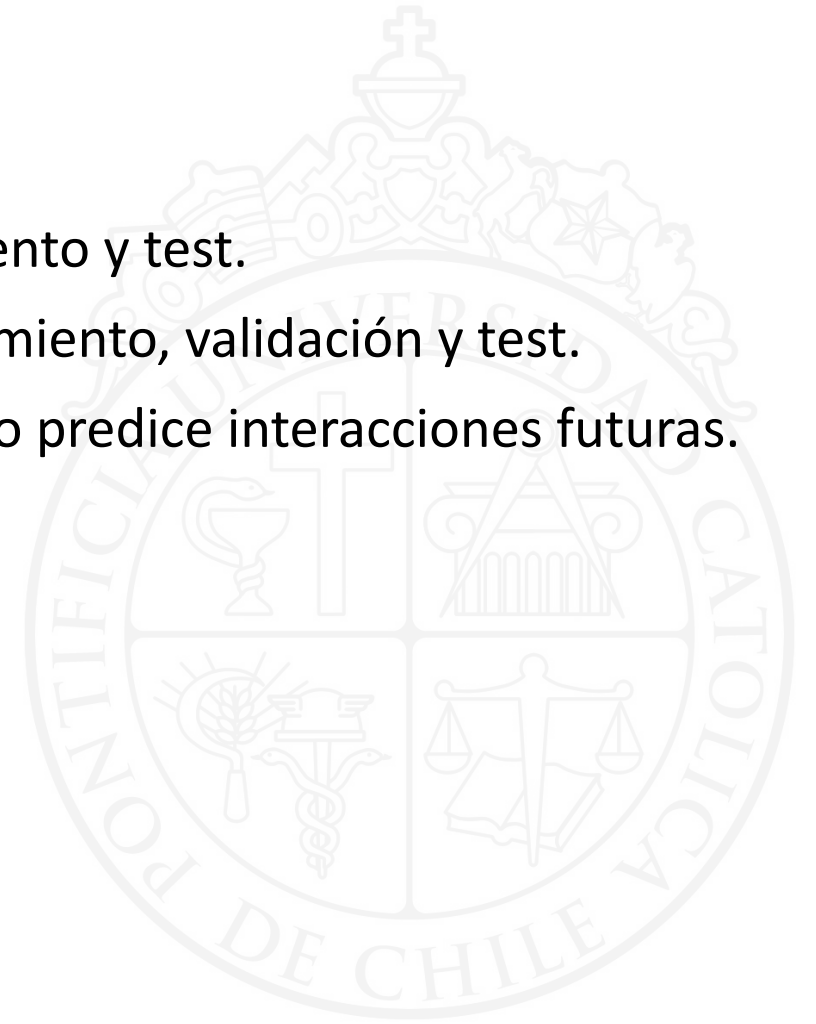


¿Qué problemas nos puede traer hacer partición aleatoria del set de entrenamiento , validación y test para recomendación?



Consideraciones de partición de sets para recomendación (tradicional)

1. Los mismos usuarios se tienen que repetir en entrenamiento y test.
2. Todos los ítems del catálogo tienen que estar en entrenamiento, validación y test.
3. Tenemos que cortar por fecha para simular que el modelo predice interacciones futuras.



Métricas para medir Calidad de Predicción (Error)

Mean Absolute Error

$$MAE = \frac{1}{N} \sum_{i=1}^N |\hat{r}_{ui} - r_{ui}|$$

Mean Square Error

$$MSE = \frac{1}{N} \sum_{i=1}^N (\hat{r}_{ui} - r_{ui})^2$$

Root Mean Square Error

$$RMSE = \sqrt{\frac{1}{N} \sum_{i=1}^N (\hat{r}_{ui} - r_{ui})^2}$$

donde

N : cantidad de pares (usuario, item) datos evaluados

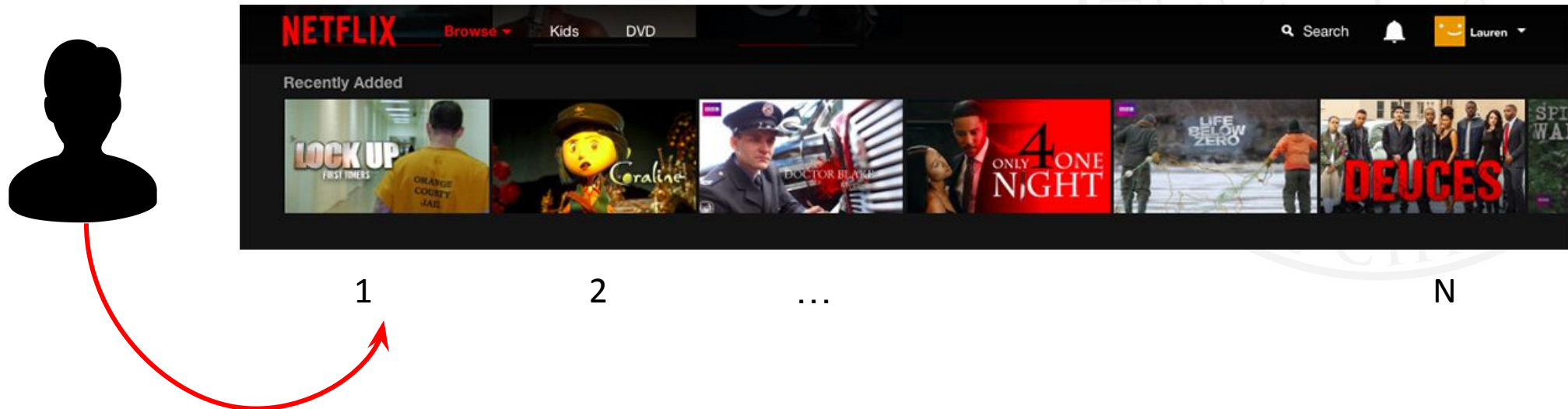
$\hat{r}_{ui}$: predicción del rating del usuario u al item i

r_{ui} : valor del rating que el usuario u le asignó al item i

Nuevo Problema de Recomendación: Ranking

- En el problema original, intentamos predecir Ratings
 - Debilidades:
 - ¿De qué nos sirve predecir ratings si no todos dan ratings?
 - ¿Cómo le sacamos provecho al feedback implícito?

Nueva versión del problema: recomendar una lista ordenada de ítems (ranking)



Visto de otra forma para evaluación...

Usuario:

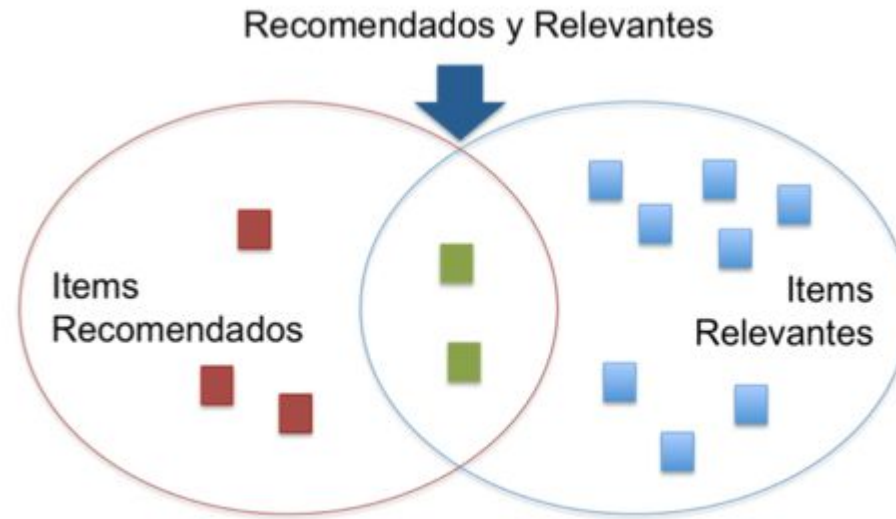
Item 12 ,	item 32 ,	item 2 ,	item 24	, item 283,	item 5
enero	enero	febrero	febrero	marzo	marzo

Queremos que estos ítems
aparezcan en la lista de las
recomendaciones

Evaluación de una lista relevante

Items Recomendados:

Son los items que me sugiere el modelo en un determinado orden de más a menos relevante.



Ítems Relevantes:

Yo conozco de mis datos de entrenamiento que estos ítems son relevantes para el usuario.

Son mis etiquetas con las que entreno el modelo

$$\text{Precisión} = \frac{|\text{Recomendados} \cap \text{Relevante}|}{|\text{Recomendados}|}$$

$$2 / 5$$

$$\text{Recall} = \frac{|\text{Recomendados} \cap \text{Relevantes}|}{|\text{Relevantes}|}$$

$$2 / 10$$

Ejemplo 1: *Precision & Recall*

Total relevantes: ■ x20 Item Recomendado ■ Item Relevante ■

Recomendador 1 ■ ■ ■ ■ ■ ■ ■ ■ ■ ■

corte = @10

$$\text{Precisión} = \frac{|\text{Recomendados} \cap \text{Relevante}|}{|\text{Recomendados}|} = \frac{5}{10} = 0.5$$

$$\text{Recall} = \frac{|\text{Recomendados} \cap \text{Relevantes}|}{|\text{Relevantes}|} = \frac{5}{20} = 0.25$$

Recomendador 2 ■ ■ ■ ■ ■

corte = @5

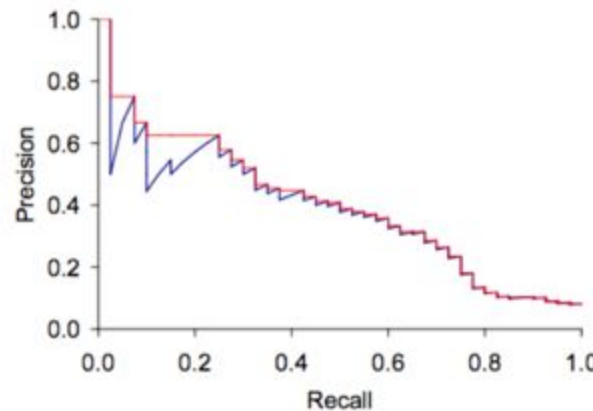
$$\text{Precisión} = \frac{|\text{Recomendados} \cap \text{Relevante}|}{|\text{Recomendados}|} = \frac{3}{5} = 0.6$$

$$\text{Recall} = \frac{|\text{Recomendados} \cap \text{Relevantes}|}{|\text{Relevantes}|} = \frac{3}{20} = 0.15$$

Voy a tener **recall máximo si recomiendo todo el catálogo** , pero voy a tener muy baja precisión porque el denominador va a ser muy grande...

Compromiso entre Precisión y Recall

- Al aumentar el *recall* (la proporción de elementos relevantes) disminuimos la precisión, por ejemplo al aumentar la cantidad de ítems relevantes.



► Figure 8.2 Precision/recall graph.

← RECALL MÁXIMO , MÍNIMA PRECISIÓN
RECOMIENDO TODO PERO A UN COSTO DE
MENOS PRECISIÓN

- Para controlar este efecto se utiliza la razón armónica o *F1 score*

$$F_{\beta=1} = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

Mean Reciprocal Ranking (MRR)

Mide qué tan bien rankeo los ítems, corresponde al inverso de la posición del primer elemento relevante

$$MRR = \frac{1}{r}$$

r : posición del 1er elemento relevante

Recomendador 1



$$MRR = \frac{1}{r} = \frac{1}{2} = 0.5$$

Recomendador 2



$$MRR = \frac{1}{r} = \frac{1}{2} = 0.5$$

en MRR son iguales.

Precision at N (P@N)

- Corresponde a la precisión en un punto específico de la lista de los ítems recomendados

$$\text{Precision@}n = \frac{1}{n} \sum_{i=1}^n \text{Rel}(i)$$

$\text{Rel}(i) = 1$ si el ítem i es relevante

Recomendador 1 ■■■■■■■■■■

$$\text{Precision@}5 = \frac{1}{n} \sum_{i=1}^n \text{Rel}(i) = \frac{2}{5} = 0.4$$

Recomendador 2 ■■■■■ XXRRR

$$\text{Precision@}5 = \frac{1}{n} \sum_{i=1}^n \text{Rel}(i) = \frac{3}{5} = 0.6$$

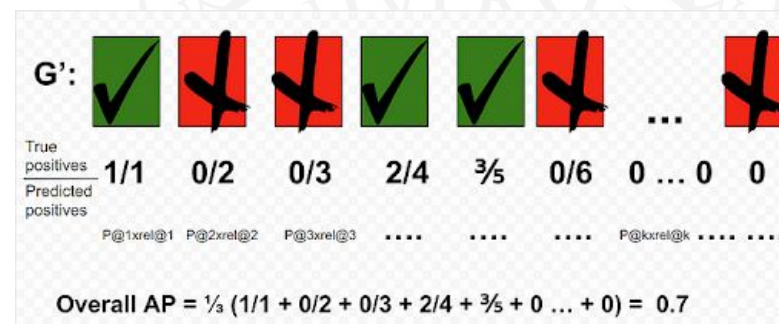
Mean Average Precision (MAP)

Average Precision (AP)

- AP se calcula promediando cada vez que encontramos un elemento relevante (*recall point*) sobre una lista única

$$AP = \frac{\sum_{k=1}^n P@i \cdot \text{rel}(k)}{|\text{Relevantes}|}$$

Rel(i) = 0 o Rel(i) = 1
p@k divide por k



Mean Average Precision (MAP)

- Considera el promedio de AP sobre un conjunto de listas recomendadas a todos los usuarios

$$MAP = \frac{1}{N} \sum_{u=1}^N AP(u)$$

N: número de usuarios o listas recomendadas

Discounted Cumulative Gain

- **DCG**: Discounted Cumulative Gain, mide la ganancia al ordenar la lista de forma correcta

$$\text{DCG}_p = \sum_{i=1}^p \frac{2^{\text{rel}_i} - 1}{\log_2(1 + i)}$$

Asume log2 ya
que tenemos:
rel = 1 o 0

- **nDCG**: Normalized Discounted Cumulative Gain

$$\text{nDCG}_p = \frac{\text{DCG}_p}{\text{iDCG}_p}$$

iDCG : es el DCG ideal
donde todos los
elementos son
relevantes.

Explicación detallada: <https://www.evidentlyai.com/ranking-metrics/ndcg-metric>

Coverage

Coverage: cuánto del dataset puedo recomendar? La idea es acercar a la gente a TODO el contenido. Si las recomendaciones están sesgadas a una proporción de X ítems , no es un buen signo.

Item Coverage

Porcentaje de ítems que son recomendados por lo menos una vez

User Coverage

Porcentaje de usuarios a los cuales se les pudo hacer una recomendación

EJ.

AP MODELO 1 = 0.4

AP MODELO 2 = 0.7

PERO LA COBERTURA DEL MODELO 1 PROVIENE DEL 70% DEL CATALOGO ES MEJOR M1

Diversidad

PAIRWISE DIVERSITY: Diversidad promedio que encuentro en el contenido de los items recomendados con respecto a todo el catálogo.

De dos recomendadores igual de buenos voy a preferir el que sea más diverso.

En RRSS si no promuevo la diversidad pueden haber FILTER BUBBLES , donde x ej personas que recomiendo solo cuentas de una postura política la idea es generar diversidad para evitar polarización.



Reflexión

Es necesario sacar métricas diferentes porque nos van a dar insights distintos

MAYOR COBERTURA?

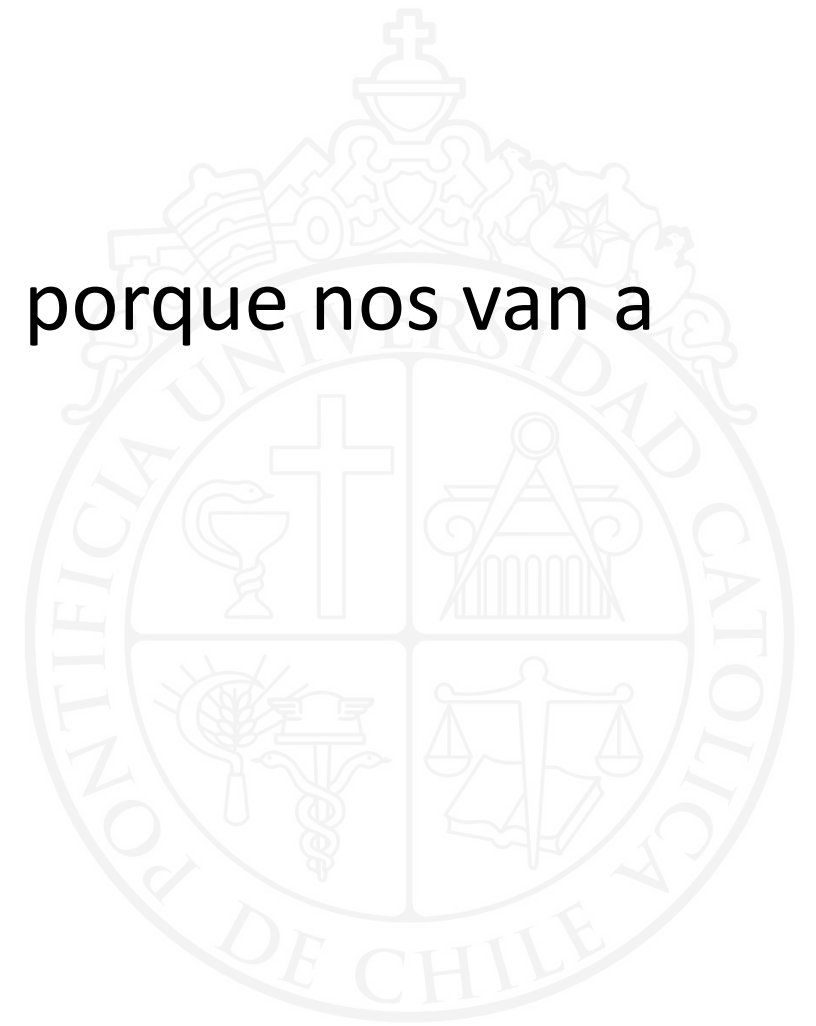
MAYOR DIVERSIDAD?

MÁS RELEVANTES EN LISTAS MÁS CORTAS

MAYOR RECALL?

MAYOR PRECISIÓN?

MAYOR DCG?



En esta clase

1. Repaso de Factorización Matricial (SVD y Funk SVD)
2. Feedback implícito
3. Optimización de recomendador con feedback implícito
4. Evaluación de modelos de ranking personalizado.



Reflexión

Hasta ahora hemos utilizado solo interacciones de usuario e ítem.

¿Cómo podemos incorporar contenido a la recomendación?

Práctico

<https://colab.research.google.com/drive/1jsSh4xGM0oB-BdIYrrvVf4MYWoiH-VjK?usp=sharing>

